

Capítulo 3

Semana 3: Seguridad y Confiabilidad en Sistemas Distribuidos

Fechas: 18 – 24 de mayo de 2026

Semana: 3 de 18

Resultado de Aprendizaje: Analiza las características y principios de los sistemas distribuidos, evaluando sus ventajas, limitaciones y retos desde una perspectiva crítica e interdisciplinaria.

3.1 Amenazas y Vulnerabilidades en Entornos Distribuidos

Los sistemas distribuidos, al operar sobre redes abiertas, presentan una superficie de ataque mucho mayor que los sistemas centralizados. El modelo de amenazas más utilizado es **STRIDE**, desarrollado por Microsoft (**kaufman2023**):

Letra	Amenaza	Descripción y ejemplo en SD
S	Spoofing	Falsificar la identidad de un nodo. Ej.: un servicio se hace pasar por la BD.
T	Tampering	Alterar datos en tránsito. Ej.: modificar un mensaje JSON en la red.
R	Repudiation	Negar haber realizado una acción. Ej.: cliente niega haber pedido algo.
I	Info Disclosure	Exponer datos confidenciales. Ej.: logs que muestran contraseñas.
D	Denial of Service	Agotar recursos. Ej.: DDoS contra el API Gateway.
E	Elevation of Privilege	Obtener permisos superiores. Ej.: inyección SQL para acceder como admin.

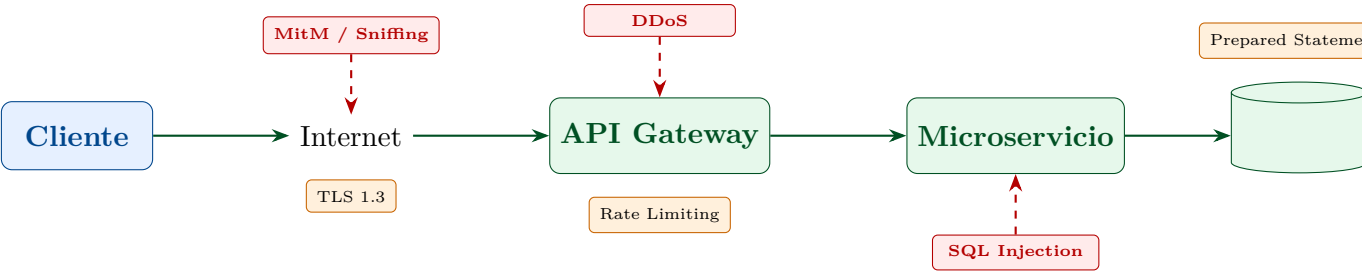


Figura 3.1: Mapa de amenazas en una arquitectura distribuida típica y sus contramedidas.

3.2 Criptografía Aplicada a Sistemas Distribuidos

3.2.1 Cifrado Simétrico vs. Asimétrico

Tipo	Claves	Velocidad	Uso en SD
Simétrico	Una sola clave compartida	Muy rápido (AES: GB/s)	Cifrado de datos en disco/canal
Asimétrico	Par público/privado	Lento (RSA: KB/s)	Intercambio de claves, firmas digitales
Híbrido	Asimétrico negocia; simétrico cifra	Alto	TLS 1.3, Signal Protocol
Hashing	Sin clave (función one-way)	Muy rápido	Integridad de mensajes, almacenamiento de contraseñas

3.2.2 Implementación en Java: Cifrado AES y HMAC

Guía Práctica IDE – IntelliJ IDEA – Proyecto Maven: seguridad-distribuida

Paso 1: Crear proyecto Maven `seguridad-distribuida` con Java 21.

Paso 2: Agregar dependencia en `pom.xml` :

```
1 <dependency>
2   <groupId>org.bouncycastle</groupId>
3   <artifactId>bcprov-jdk18on</artifactId>
4   <version>1.78.1</version>
5 </dependency>
```

Paso 3: Crear el paquete `ec.edu.uteq.distribuidas.seguridad` y la clase `CifradoAES.java`:

```
1 package ec.edu.uteq.distribuidas.seguridad;
2
3 import javax.crypto.*;
4 import javax.crypto.spec.*;
5 import java.security.*;
6 import java.util.Base64;
7
8 /**
9  * Implementacion de cifrado simetrico AES-256-GCM.
10  * GCM (Galois/Counter Mode) proporciona autenticacion
11  * integrada (AEAD).
12  * Es el modo recomendado para sistemas distribuidos en 2026.
13  */
14 public class CifradoAES {
15
16     private static final String ALGORITMO = "AES/GCM/NoPadding"
17         ;
18     private static final int BITS_CLAVE = 256;
19     private static final int LONGITUD_IV = 12;    // 96 bits
20         recomendado para GCM
21     private static final int BITS_TAG = 128;    // tag de
22         autenticacion GCM
23
24     /** Genera una clave AES-256 aleatoria y criptograficamente
25         segura. */
26     public static SecretKey generarClave() throws
27         NoSuchAlgorithmException {
28         KeyGenerator generador = KeyGenerator.getInstance("AES"
29             );
30         generador.init(BITS_CLAVE, new SecureRandom());
31         return generador.generateKey();
32     }
33
34     /**
35      * Cifra un mensaje con AES-256-GCM.
36      *
37      * @param textoPlano texto a cifrar
```

```
31     * @param clave         clave AES-256
32     * @return arreglo de bytes: [IV (12 bytes) | ciphertext+
33     *                               tag]
34     */
35     public static byte[] cifrar(String textoPlano, SecretKey
36         clave)
37         throws GeneralSecurityException {
38         // Generar IV aleatorio unico por cada cifrado
39         byte[] iv = new byte[LONGITUD_IV];
40         new SecureRandom().nextBytes(iv);
41
42         Cipher cifrador = Cipher.getInstance(ALGORITMO);
43         GCMParameterSpec parametros = new GCMParameterSpec(
44             BITS_TAG, iv);
45         cifrador.init(Cipher.ENCRYPT_MODE, clave, parametros);
46
47         byte[] ciphertext = cifrador.doFinal(
48             textoPlano.getBytes(java.nio.charset.
49                 StandardCharsets.UTF_8));
50
51         // Concatenar IV + ciphertext para transmision
52         byte[] resultado = new byte[LONGITUD_IV + ciphertext.
53             length];
54         System.arraycopy(iv, 0, resultado, 0, LONGITUD_IV);
55         System.arraycopy(ciphertext, 0, resultado, LONGITUD_IV,
56             ciphertext.length);
57         return resultado;
58     }
59
60     /**
61     * Descifra un mensaje cifrado con AES-256-GCM.
62     * Si el mensaje fue alterado, GCM lanzara
63     * AEADBadTagException.
64     */
65     public static String descifrar(byte[] mensajeCifrado,
66         SecretKey clave)
67         throws GeneralSecurityException {
68         // Extraer IV de los primeros 12 bytes
69         byte[] iv = new byte[LONGITUD_IV];
70         System.arraycopy(mensajeCifrado, 0, iv, 0, LONGITUD_IV)
71         ;
```

```
63
64     byte[] ciphertext = new byte[mensajeCifrado.length -
        LONGITUD_IV];
65     System.arraycopy(mensajeCifrado, LONGITUD_IV,
        ciphertext, 0, ciphertext.length);
66
67     Cipher descifrador = Cipher.getInstance(ALGORITMO);
68     GCMParameterSpec parametros = new GCMParameterSpec(
        BITS_TAG, iv);
69     descifrador.init(Cipher.DECRYPT_MODE, clave, parametros
        );
70
71     byte[] textoPlano = descifrador.doFinal(ciphertext);
72     return new String(textoPlano, java.nio.charset.
        StandardCharsets.UTF_8);
73 }
74
75 /** Convierte bytes a Base64 para transmision como texto.
76     */
77 public static String aBase64(byte[] datos) {
78     return Base64.getEncoder().encodeToString(datos);
79 }
80
81 public static byte[] deBase64(String base64) {
82     return Base64.getDecoder().decode(base64);
83 }
84
85 // Demo
86 public static void main(String[] args) throws Exception {
87     System.out.println("=== Demo AES-256-GCM ===\n");
88
89     SecretKey clave = generarClave();
90     System.out.println("Clave generada (Base64): "
        + aBase64(clave.getEncoded()).substring(0, 20) + "
        ...");
91
92     String mensajeOriginal = "Token JWT del usuario:
        eyJhbGciOiJSUzI1NiJ9...";
93     System.out.println("Mensaje original: " +
        mensajeOriginal);
94
```

```
95      // Cifrar
96      byte[] cifrado = cifrar(mensajeOriginal, clave);
97      System.out.println("Mensaje cifrado (Base64): "
98          + aBase64(cifrado).substring(0, 40) + "...");
99
100     // Descifrar
101     String descifrado = descifrar(cifrado, clave);
102     System.out.println("Mensaje descifrado: " + descifrado)
103         ;
104
105     System.out.println("\nIntegridad verificada: "
106         + mensajeOriginal.equals(descifrado));
107
108     // Simular alteracion del ciphertext
109     System.out.println("\n--- Simulando ataque de
110         alteracion ---");
111     byte[] alterado = cifrado.clone();
112     alterado[15] ^= 0xFF; // alterar un byte del
113         ciphertext
114     try {
115         descifrar(alterado, clave);
116     } catch (AEADBadTagException e) {
117         System.out.println("ALERTA: Mensaje alterado
118             detectado! " +
119                 e.getClass().getSimpleName());
120     }
121 }
```

Listing 3.1: CifradoAES.java – AES-GCM con generación de clave segura

3.2.3 HMAC: Autenticación de Mensajes sin Cifrado

En ocasiones no se necesita confidencialidad sino solo **integridad y autenticidad** de los mensajes. Para eso se usa HMAC (Hash-based Message Authentication Code):

```
1 package ec.edu.uteq.distribuidas.seguridad;
2
3 import javax.crypto.Mac;
4 import javax.crypto.spec.SecretKeySpec;
5 import java.security.InvalidKeyException;
```

```
6 import java.security.NoSuchAlgorithmException;
7 import java.util.Base64;
8 import java.util.Arrays;
9
10 /**
11  * Implementacion de HMAC-SHA256 para autenticacion de mensajes
12  * .
13  * Usado en: JWT firma, webhooks, APIs REST con firma.
14  */
15 public class HMAC {
16
17     private static final String ALGORITMO = "HmacSHA256";
18
19     /**
20      * Calcula el HMAC-SHA256 de un mensaje.
21      *
22      * @param mensaje datos a firmar (como bytes UTF-8)
23      * @param claveSec clave secreta compartida entre emisor y
24      * receptor
25      * @return firma HMAC de 32 bytes
26      */
27     public static byte[] firmar(String mensaje, byte[] claveSec
28         )
29         throws NoSuchAlgorithmException,
30             InvalidKeyException {
31         Mac mac = Mac.getInstance(ALGORITMO);
32         SecretKeySpec claveSpec = new SecretKeySpec(claveSec,
33             ALGORITMO);
34         mac.init(claveSpec);
35         return mac.doFinal(mensaje.getBytes(java.nio.charset.
36             StandardCharsets.UTF_8));
37     }
38
39     /**
40      * Verifica la firma HMAC de un mensaje.
41      * Usa comparacion en tiempo constante para evitar timing
42      * attacks.
43      */
44     public static boolean verificar(String mensaje, byte[]
45         firmaEsperada,
46
47         byte[] claveSec)
```

```
39         throws NoSuchAlgorithmException,
           InvalidKeyException {
40     byte[] firmaCalculada = firmar(mensaje, claveSec);
41     // MessageDigest.isEqual hace comparacion en tiempo
           constante
42     return java.security.MessageDigest.isEqual(
           firmaCalculada, firmaEsperada);
43 }
44
45 public static void main(String[] args) throws Exception {
46     // Clave compartida (en produccion: derivar con PBKDF2
           o HKDF)
47     byte[] claveSecreta = "s3cr3t0-UTEQ-2026-SD!".getBytes
           ();
48
49     String payload = "{\"userId\":\"42\",\"role\":\"
           estudiante\",\"exp\":1800000}";
50     System.out.println("Payload: " + payload);
51
52     // Firmar
53     byte[] firma = firmar(payload, claveSecreta);
54     String firmaB64 = Base64.getEncoder().encodeToString(
           firma);
55     System.out.println("Firma HMAC-SHA256: " + firmaB64);
56
57     // Verificar (caso correcto)
58     boolean valido = verificar(payload, firma, claveSecreta
           );
59     System.out.println("Verificacion (original): " + valido
           );
60
61     // Verificar (payload alterado)
62     String payloadAlterado = payload.replace("estudiante",
           "admin");
63     boolean invalido = verificar(payloadAlterado, firma,
           claveSecreta);
64     System.out.println("Verificacion (alterado): " +
           invalido);
65 }
66 }
```


Listing 3.2: HMAC.java – Firma y verificación de mensajes

3.3 Tolerancia a Fallos: Detección y Recuperación

3.3.1 Clasificación de Fallos en Sistemas Distribuidos

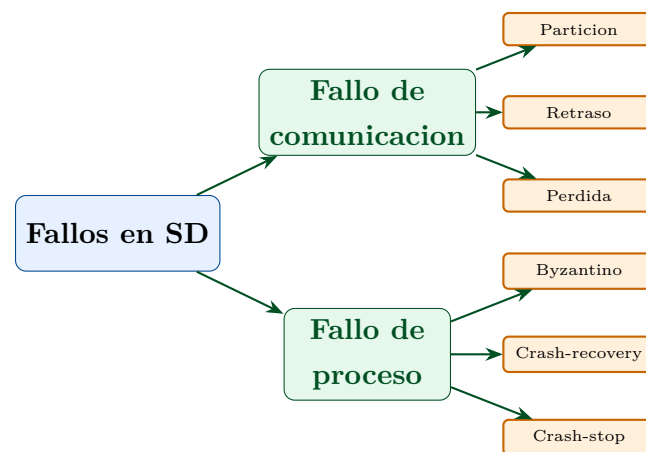


Figura 3.2: Taxonomía de fallos en sistemas distribuidos.

3.3.2 Implementación de Heartbeat y Failover en Java

```
1 package ec.edu.uteq.distribuidas.confiableidad;
2
3 import java.net.*;
4 import java.time.*;
5 import java.util.concurrent.*;
6 import java.util.function.Consumer;
7 import java.util.logging.Logger;
8
9 /**
10  * Monitorea la disponibilidad de nodos remotos mediante
11  * heartbeat.
12  * Si un nodo no responde en el umbral configurado, dispara una
13  * accion.
14  *
15  * Implementa el patron Fail-Fast: detectar el fallo
16  * rapidamente y actuar.
17  */
```

```

15 public class MonitorHeartbeat {
16
17     private static final Logger log =
18         Logger.getLogger(MonitorHeartbeat.class.getName());
19
20     private final String host;
21     private final int puerto;
22     private final int timeoutMs;
23     private final int intervaloMs;
24     private final Consumer<Boolean> callbackEstado;
25
26     private volatile boolean ultimoEstado = true;
27     private Instant ultimoHeartbeat = Instant.now();
28     private ScheduledFuture<?> tarea;
29
30     /**
31      * @param host      IP o hostname del nodo a monitorear
32      * @param puerto    puerto TCP a verificar
33      * @param timeoutMs timeout de conexion en ms
34      * @param intervaloMs intervalo entre heartbeats en ms
35      * @param callback  funcion llamada con true=activo /
36                      false=caido
37      */
38     public MonitorHeartbeat(String host, int puerto, int
39                             timeoutMs,
40                             int intervaloMs, Consumer<Boolean>
41                             callback) {
42
43         this.host = host;
44         this.puerto = puerto;
45         this.timeoutMs = timeoutMs;
46         this.intervaloMs = intervaloMs;
47         this.callbackEstado = callback;
48     }
49
50     /** Inicia el monitoreo en un hilo programado. */
51     public void iniciar(ScheduledExecutorService scheduler) {
52         tarea = scheduler.scheduleAtFixedRate(
53             this::verificarNodo, 0, intervaloMs, TimeUnit.
54                 MILLISECONDS);
55         log.info(String.format("Monitor iniciado para %s:%d " +
56             "(timeout=%dms, intervalo=%dms)",

```

```
52         host, puerto, timeoutMs, intervaloMs));
53     }
54
55     public void detener() {
56         if (tarea != null) tarea.cancel(false);
57     }
58
59     private void verificarNodo() {
60         boolean activo = false;
61         try (Socket socket = new Socket()) {
62             socket.connect(
63                 new InetSocketAddress(host, puerto),
64                 timeoutMs
65             );
66             activo = true;
67             ultimoHeartbeat = Instant.now();
68         } catch (Exception e) {
69             log.warning(String.format("Heartbeat fallido para %
70                                     s:%d - %s",
71                                     host, puerto, e.getMessage()));
72         }
73
74         // Notificar cambio de estado
75         if (activo != ultimoEstado) {
76             ultimoEstado = activo;
77             String estado = activo ? "RECUPERADO" : "CAIDO";
78             System.out.printf("[%s] Nodo %s:%d -> %s%n",
79                               LocalDateTime.now(), host, puerto, estado);
80             callbackEstado.accept(activo);
81         }
82     }
83
84     public boolean estaActivo() { return ultimoEstado; }
85     public Instant getUltimoHeartbeat() { return
86         ultimoHeartbeat; }
87
88     /** Demo con dos nodos: uno real (servidor de la Semana 2)
89         y uno falso. */
90     public static void main(String[] args) throws
91         InterruptedException {
92         ScheduledExecutorService scheduler =
```

```
89         Executors.newScheduledThreadPool(4);
90
91         // Monitor para el servidor TCP de la Semana 2
92         MonitorHeartbeat monitorReal = new MonitorHeartbeat(
93             "localhost", 9000, 1000, 2000,
94             activo -> {
95                 if (!activo) {
96                     System.out.println("[FAILOVER] Activando
97                         nodo de respaldo...");
98                     // Aqui iria la logica de failover
99                 }
100             }
101         );
102
103         // Monitor para un host inexistente (siempre falla)
104         MonitorHeartbeat monitorFalso = new MonitorHeartbeat(
105             "192.168.99.99", 9000, 500, 3000,
106             activo -> System.out.println("[FAILOVER] Nodo
107                 99.99: " + activo)
108         );
109
110         monitorReal.iniciar(scheduler);
111         monitorFalso.iniciar(scheduler);
112
113         // Correr 15 segundos
114         Thread.sleep(15_000);
115         monitorReal.detener();
116         monitorFalso.detener();
117         scheduler.shutdown();
118
119         System.out.println("Estado final nodo real: " +
120             monitorReal.estaActivo());
121     }
122 }
```

Listing 3.3: MonitorHeartbeat.java – Deteccion de fallos por latido

3.4 Acuerdos de Nivel de Servicio (SLA)

Definición: SLA – Service Level Agreement

Un **SLA** es el contrato formal entre un proveedor de servicios y un cliente que define los niveles de calidad comprometidos: disponibilidad, latencia, throughput y los procedimientos de penalización por incumplimiento (van Steen & Tanenbaum, 2023).

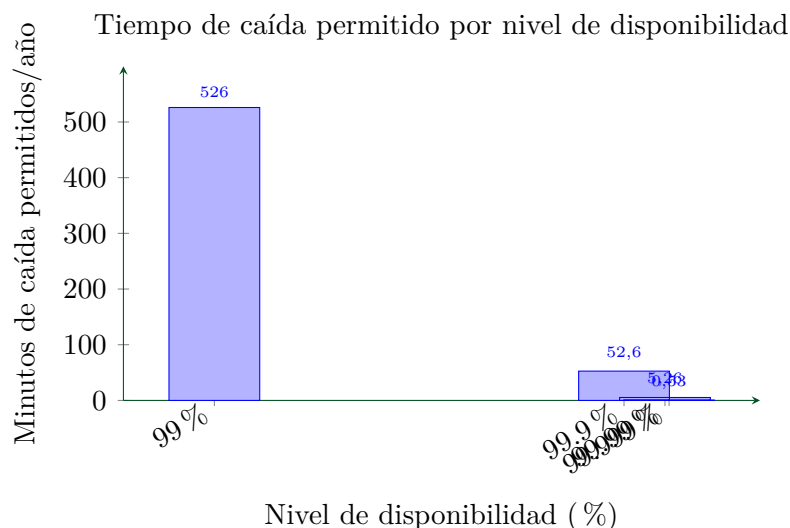


Figura 3.3: Minutos de caída permitidos por año según nivel de disponibilidad SLA.

Ejercicio Resuelto: Calcular métricas de SLA para el sistema bancario Banecuador

El sistema de créditos de Banecuador tiene los siguientes datos del año 2025:

- Total de incidentes: 3
- Tiempos de caída: 45 min, 120 min, 18 min
- El SLA comprometido es de 99.95 % de disponibilidad anual.

Calcule: (a) disponibilidad real alcanzada; (b) si se cumplió el SLA; (c) el tiempo total de penalización según la cláusula “por cada hora de incumplimiento se descuenta el 5 % de la factura mensual de \$12.000”.

Solución:

(a) Total de caída: $45 + 120 + 18 = 183$ minutos = 3,05 h.

Horas en el año: $365 \times 24 = 8,760$ h.

$$D_{\text{real}} = \frac{8,760 - 3,05}{8,760} \times 100 = 99,9652 \%$$

(b) SLA comprometido: 99.95 %. Tiempo permitido: $0,0005 \times 8,760 \times 60 = 262,8$

min.

Tiempo real de caída: $183 \text{ min} < 262.8 \text{ min}$.

SLA cumplido: la disponibilidad real (99.965 %) supera el comprometido (99.95 %).

(c) No aplican penalizaciones en este caso pues el SLA se cumplió.

Si el incidente de 120 min hubiera sido de 300 min, el total sería $45 + 300 + 18 = 363$ min, superando el límite en $363 - 262,8 = 100,2 \text{ min} \approx 1,67 \text{ h}$ de incumplimiento.

Penalización: $1,67 \times 5 \% \times \$12,000 = \$1,002$.

3.4.1 Práctica de Clase – Semana 3

Actividad Práctica: Análisis de Caso: Fallo en Sistema Distribuido Real (FORM-AC-01)

Caso asignado: Incidente de CrowdStrike (19 de julio de 2024) — el mayor apagón informático de la historia hasta esa fecha.

Instrucciones:

1. Leer el postmortem oficial de CrowdStrike y los reportes de Microsoft disponibles públicamente.
2. Identificar: tipo de fallo (STRIDE), componentes afectados (sistemas bancarios, aerolíneas, hospitales en Ecuador).
3. Analizar qué mecanismos de tolerancia a fallos fallaron o no existían.
4. Proponer al menos **cinco** mejoras técnicas con justificación (cifrado, heartbeat, canary releases, circuit breaker, etc.).

Entregable: Informe académico individual TA-IND-01 (máx. 4 páginas, APA 7ma edición, al menos 3 referencias científicas).